UPB-CIENTÍFICA

Documento de Arquitectura del Sistema Distribuido

Diagrama de Despliegue · Referencia técnica

Sistemas Distribuidos 2026-20

Facultad de Ingeniería de Sistemas e Informática

Universidad Pontificia Bolivariana — Seccional Bucaramanga

Proyecto de aula

Centro de Computación Avanzada (CCA)

Propósito de este documento

Este documento explica el diagrama de despliegue del sistema UPB-CIENTÍFICA: qué hace cada nodo, cómo se comunican entre sí, por qué se tomó cada decisión de arquitectura y qué supuestos maneja el diseño que no quedan dibujados en el diagrama.

Es la referencia que debe leer cualquier integrante del equipo **antes** de escribir código en su módulo. Un diagrama muestra la estructura; este documento explica el razonamiento detrás de esa estructura, que es lo que permite tomar decisiones coherentes durante el desarrollo.

1. Restricciones del enunciado

El diseño parte de un conjunto de reglas que no son negociables. Conviene tenerlas presentes porque condicionan casi todas las decisiones posteriores.

Regla	Dónde se cumple en la arquitectura
Tecnologías de la Figura 1: PHP/SOAP, JavaScript, Java/RMI, Go/gRPC, MPI	Un nodo dedicado por cada tecnología
Toda la infraestructura en el Centro de Computación Avanzada	Las siete máquinas virtuales son del CCA
Sin entornos basados en contenedores	Entornos nativos: Debian, JVM, Node.js, Go, Apache
Sin servicios Cloud de terceros	No hay proveedores externos en ningún nodo
Clúster sobre máquinas del laboratorio	Recinto «Clúster HPC — Máquinas del Laboratorio»
Componentes integrables como software libre	Debian, OpenLDAP, NFS, PostgreSQL, FFmpeg, GnuPG, Prometheus, Grafana
Perfiles administrados por servicio de directorio	OpenLDAP en `node-soap`
Cifrado asimétrico de respaldos con GPG	`respaldo-gpg.sh` y `upbc-respaldo.pub`
Autenticación con token o múltiple factor	`mfa.php` y `credencial.local` en los clientes
Monitoreo web con histórico y alertas	Prometheus y Grafana en `node-monitoreo`

Asignación de tecnologías

La Figura 1 del enunciado etiqueta cada servidor de aplicación con su función. La correspondencia es:

- **SOAP / PHP** → usuarios y autenticación
- **JavaScript** → servidor web y entrega de la interfaz
- **Java / RMI** → gestor del clúster HPC
- **Go / gRPC** → servicio de sincronización de archivos
- **MPI** → ejecución paralela en las máquinas del laboratorio

2. Vista general

Siete máquinas virtuales alojadas en el CCA, más el clúster físico del laboratorio. El principio que ordena toda la topología es que **`node-web` es la única puerta de los clientes hacia los servicios del producto**: el navegador nunca habla directamente con SOAP, RMI, el repositorio ni el clúster. Las dos excepciones son `node-sync`, que recibe conexiones gRPC directas de los clientes nativos, y la autenticación del agente de escritorio contra el servicio SOAP.

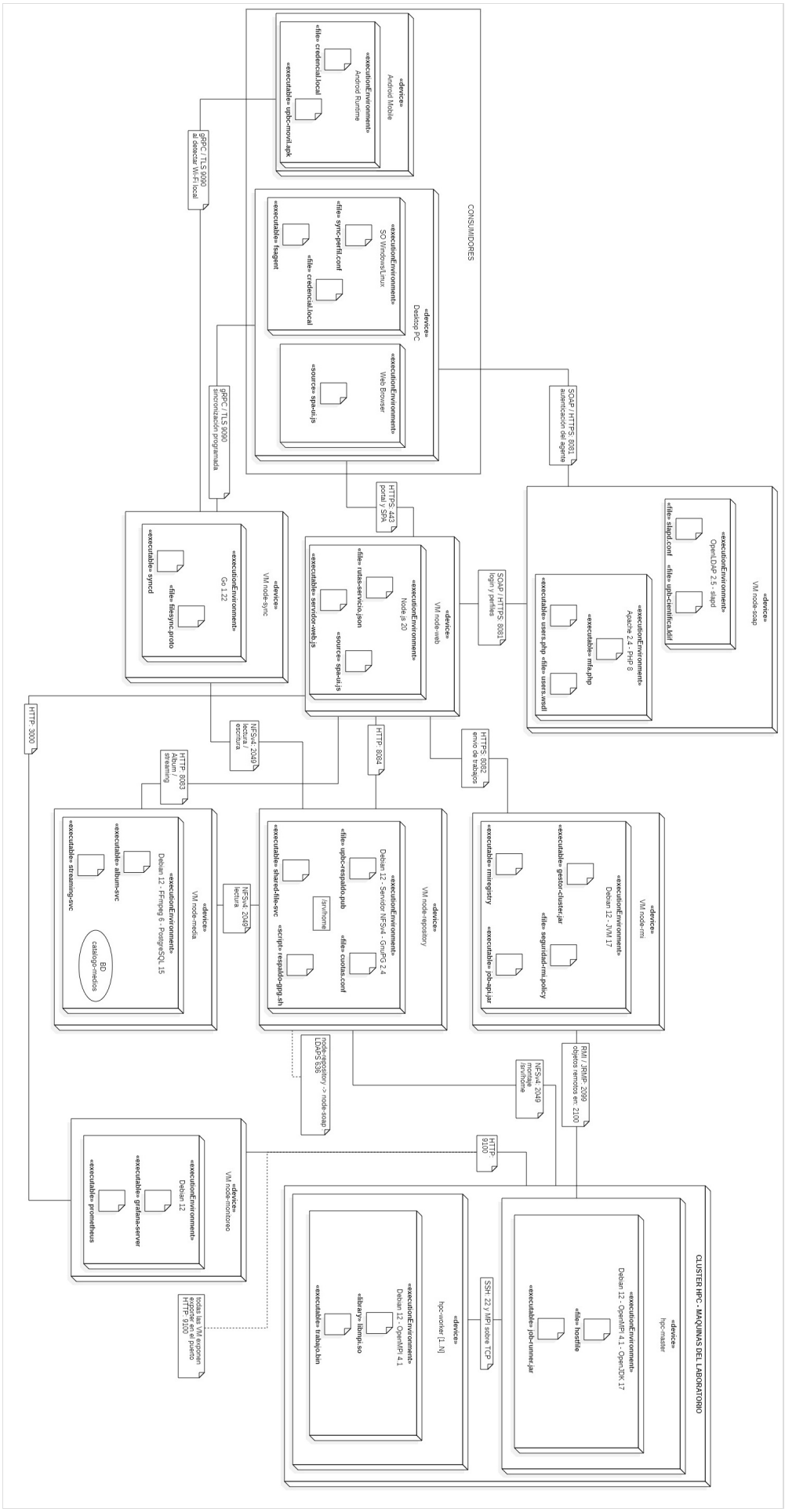
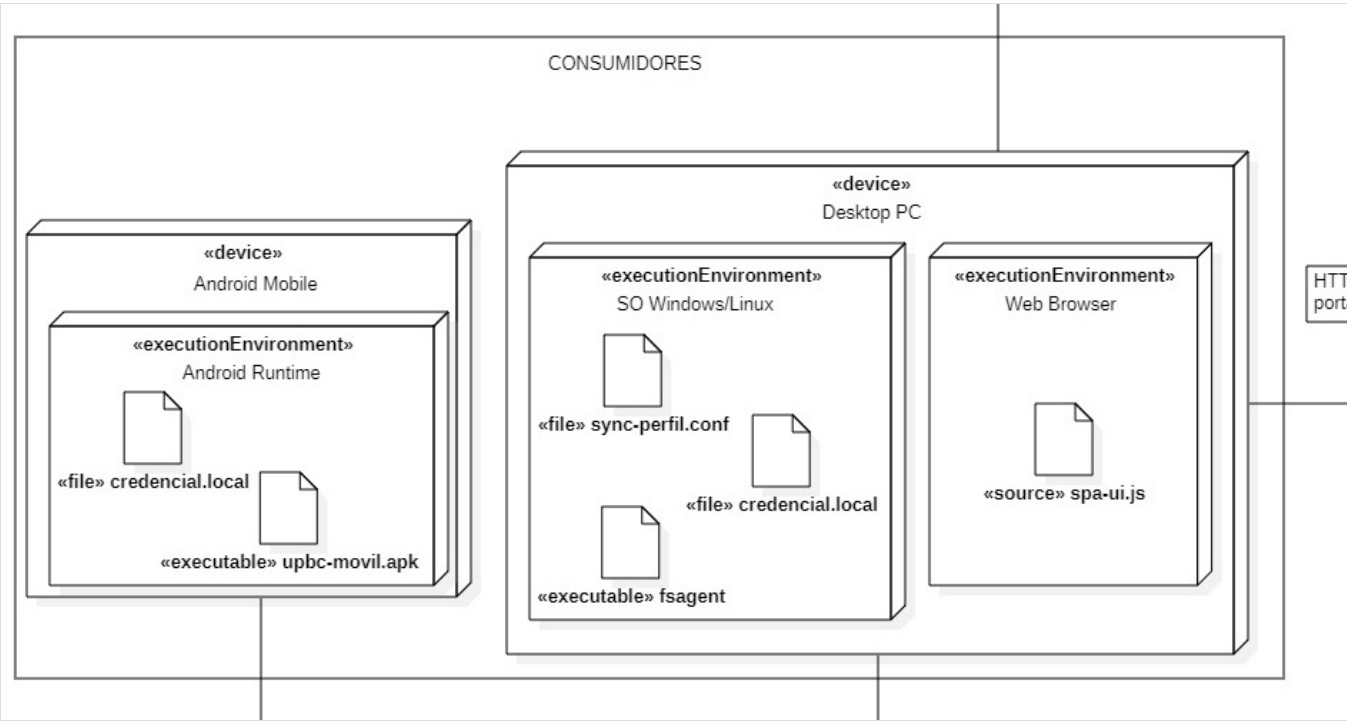


Diagrama de despliegue completo — girado 90° para su lectura. Los recortes por nodo se detallan en la sección 3.

3. Inventario de nodos

3.1 Consumidores



Clientes del sistema: equipo de escritorio y terminal móvil.

Desktop PC — Windows / Linux

Convive con dos entornos de ejecución en la misma máquina:

- **Navegador web:** ejecuta `spa-ui.js`, la interfaz gráfica completa del sistema.
- **Sistema operativo:** ejecuta `fsagent`, el agente de sincronización.

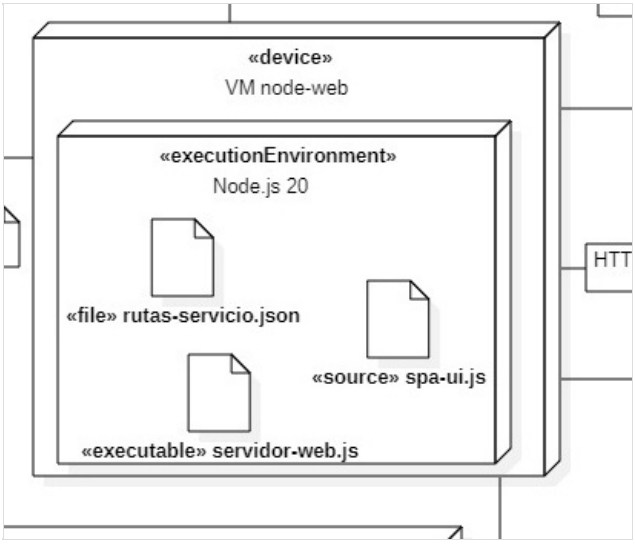
`fsagent` es un proceso en segundo plano que vigila un directorio local y sincroniza según el horario definido en `sync-perfil.conf`. No es una aplicación de escritorio completa: necesita una ventana pequeña de configuración —inicio de sesión, selección de carpeta y horario— pero el grueso del esfuerzo de desarrollo está en la lógica de sincronización, no en la interfaz. El archivo `credencial.local` almacena el token de sesión.

Android Mobile

Ejecuta `upbc-movil.apk` sobre Android Runtime, con su propio `credencial.local`. A diferencia del escritorio, **el móvil no usa horario programado**: dispara la sincronización al detectar la red Wi-Fi local donde reside el servidor, tal como exige el requisito de File Sync.

3.2 Servidores de aplicación

VM node-web — Node.js 20



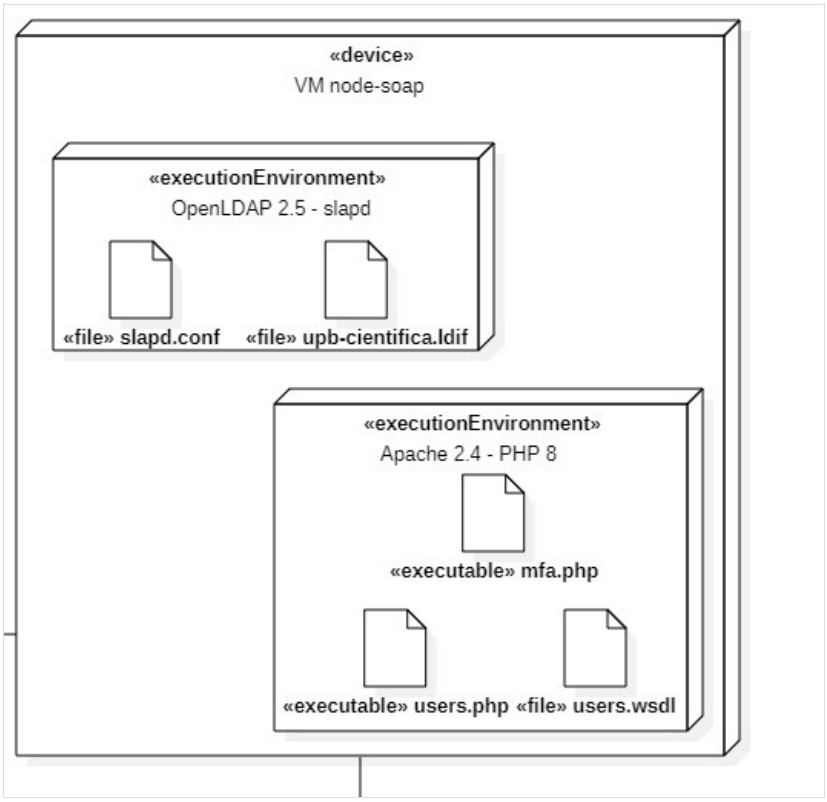
Servidor web: entrega la interfaz y orquesta los servicios internos.

Es el orquestador del sistema. Sirve la interfaz al navegador y expone la API que ésta consume. Por cada petición del cliente valida la sesión contra SOAP y llama al servicio interno correspondiente.

- `servidor-web.js` — servidor y API
- `spa-ui.js` — bundle de la interfaz que se entrega al navegador
- `rutas-servicio.json` — mapa de a qué nodo se dirige cada operación

No almacena nada. Todo dato persistente vive en `node-repository`.

VM node-soap — OpenLDAP 2.5 y Apache 2.4 / PHP 8



Nodo de identidad: directorio LDAP y fachada SOAP en la misma máquina.

Contiene **dos entornos de ejecución** sobre el mismo servidor:

- **OpenLDAP (`slapd`):** `upb-cientifica.ldif` con usuarios, grupos, uid, gid, cuota y mapa de servicios; `slapd.conf` con esquemas, listas de control de acceso y TLS.

- **Apache y PHP:** `users.php` como servicio SOAP de usuarios, descrito en `users.wsdl`, y `mfa.php` para el segundo factor de autenticación.

> **Sobre el nombre.** `node-soap` describe solo la mitad de lo que hace este nodo. Se conserva para que el equipo identifique con rapidez dónde está el código PHP, pero **funcionalmente es el servidor de identidad completo**: SOAP es la fachada y LDAP es la base de datos que hay detrás. SOAP consulta el directorio por *loopback*, de modo que la autenticación nunca cruza la red.

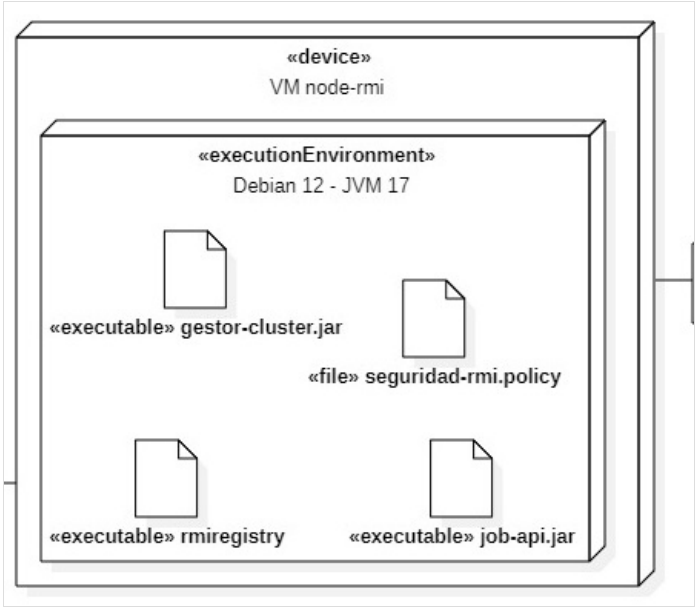
VM node-sync — Go 1.22

Servidor gRPC de sincronización de archivos.

- `syncd` — servidor gRPC sobre TLS
- `filesync.proto` — contrato Protocol Buffers

Recibe los cambios de `fsagent` y de la aplicación móvil y los escribe en el Home del usuario, montado por NFS. Es el único servicio que acepta conexiones directas de clientes sin pasar por `node-web`, porque gRPC sobre HTTP/2 no tiene sentido intermediarlo a través de una API REST.

VM node-rmi — JVM 17

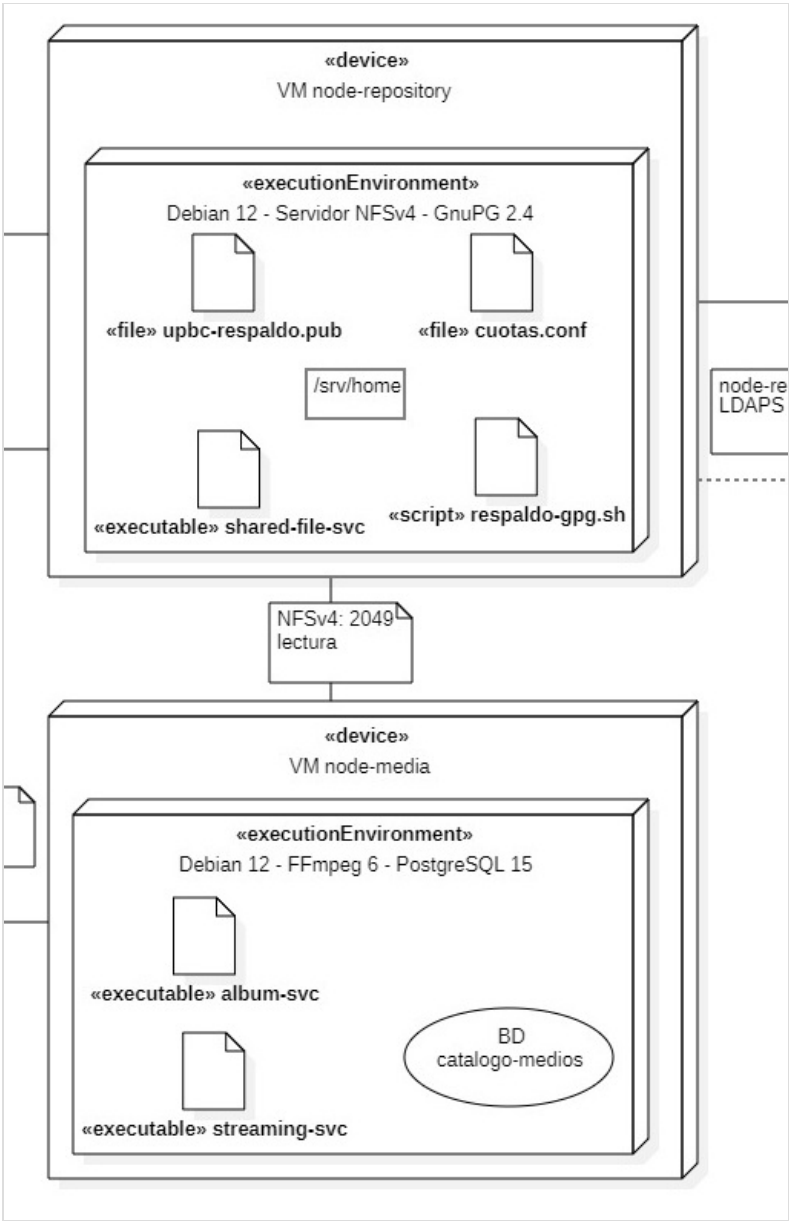


Gestor del clúster: único punto de entrada hacia el HPC.

- `rmiregistry` — servicio de nombrado
- `gestor-cluster.jar` — objeto remoto `JobService`: cola de trabajos y callback de estado
- `job-api.jar` — fachada HTTP para que `node-web` envíe trabajos sin hablar RMI
- `seguridad-rmi.policy` — gestor de seguridad

> **Detalle crítico de implementación.** Por defecto, RMI exporta los objetos remotos en un puerto aleatorio, lo que hace imposible abrir el firewall del CCA de forma acotada. Debe fijarse desde el primer día: `UnicastRemoteObject.exportObject(obj, 2100)`. Es el error que más tiempo suele consumir en proyectos basados en RMI.

3.3 Implementación de servicios



Repositorio de archivos y nodo de medios, conectados por NFSv4.

VM node-repository — Debian 12, servidor NFSv4 y GnuPG 2.4

Es el núcleo de datos del sistema. **Si se cae, se cae casi todo**: cinco componentes dependen de él.

- `/srv/home` — Home único por usuario: archivos, fotos, videos, código y conjuntos de datos
- `cuotas.conf` — cuota máxima parametrizada por usuario
- `shared-file-svc` — carga por formulario, permisos Unix y compartición entre usuarios
- `respaldo-gpg.sh` y `upbc-respaldo.pub` — respaldo cifrado

El almacenamiento se accede de dos formas distintas, y cada una tiene su razón:

1. **Por montaje NFSv4.** Lo usan `node-sync`, `node-media` y el clúster. Es nativo, rápido y preserva los permisos Unix que exige el enunciado.
2. **Por HTTP contra `shared-file-svc`.** Lo usa `node-web`. El navegador no puede montar NFS, y compartir un archivo con otro usuario es lógica de aplicación, no de sistema de archivos.

> **La llave privada de GPG no puede vivir en esta máquina.** El script cifra con la llave pública; si la privada estuviera en el mismo servidor, quien comprometa la máquina se llevaría los datos y la capacidad de descifrar las copias al mismo tiempo. Debe custodiarse fuera del nodo.

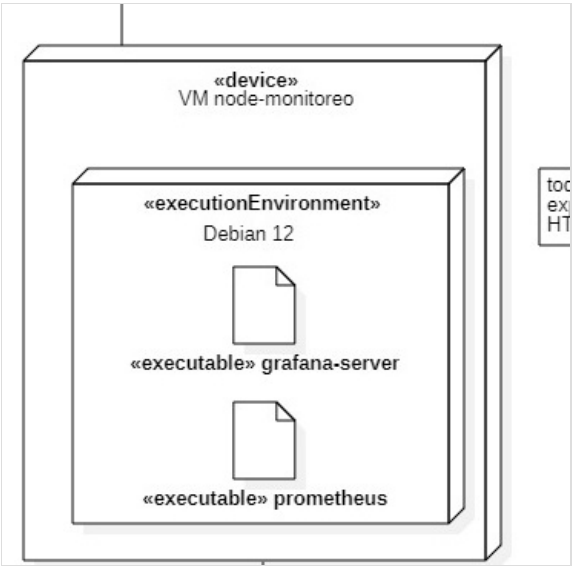
VM node-media — FFmpeg 6 y PostgreSQL 15

- `album-svc` — indexa, ordena y presenta las imágenes del Home

- `streaming-svc` — segmentado y difusión de video
- `catalogo-medios` — metadatos y miniaturas en PostgreSQL

Lee el Home por NFS **en solo lectura**. Está separado del repositorio porque el transcodificado con FFmpeg consume CPU en ráfagas, y no conviene que compita con el servidor NFS del que dependen todos los demás servicios, incluido el clúster.

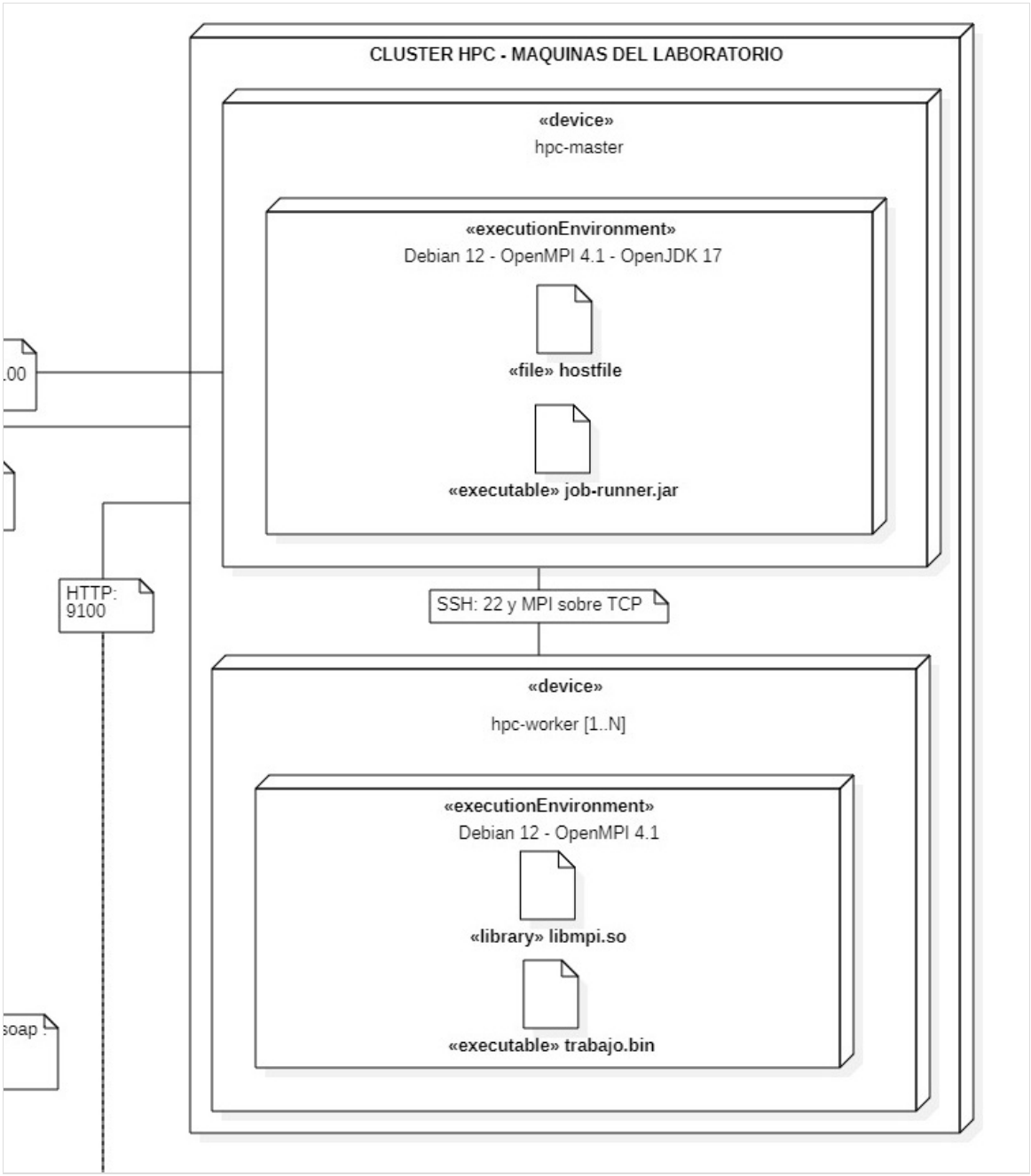
VM node-monitoreo — Prometheus y Grafana



Recolección de métricas y tableros de monitoreo.

Recolecta métricas de toda la infraestructura y las expone vía web: ocupación de CPU, memoria y almacenamiento, carga de trabajos, tiempos de ejecución y disponibilidad de servicios. Grafana aporta los reportes históricos y las alertas configurables que exige el enunciado.

3.4 Clúster HPC — máquinas del laboratorio



Nodo maestro y nodos de cómputo sobre las máquinas físicas del laboratorio.

hpc-master

- `job-runner.jar` — recibe el trabajo desde `node-rmi`, compila el fuente y lanza `mpirun`
- `hostfile` — inventario de nodos y slots disponibles

hpc-worker [1..N]

- `trabajo.bin` — binario MPI del usuario
- `libmpi.so` — runtime de OpenMPI

> **Todos los nodos montan el mismo `/srv/home`.** No es opcional: cuando `mpirun` lanza el binario en cuatro máquinas, las cuatro deben abrir el mismo conjunto de datos por la misma ruta. Si únicamente lo monta el maestro, los trabajos fallan al arrancar.

4. Rutas de comunicación

La punta de la flecha en el diagrama indica **quién inicia la conexión**.

Origen → Destino	Protocolo / puerto	Propósito
Desktop PC → `node-web`	HTTPS :443	Portal e interfaz gráfica
Desktop PC → `node-sync`	gRPC/TLS :9090	Sincronización programada
Desktop PC → `node-soap`	SOAP/HTTPS :8081	Autenticación del agente
Android → `node-sync`	gRPC/TLS :9090	Sincronización al detectar Wi-Fi local
`node-web` → `node-soap`	SOAP/HTTPS :8081	Inicio de sesión y perfiles
`node-web` → `node-rmi`	HTTPS :8082	Envío de trabajos
`node-web` → `node-media`	HTTP :8083	Álbum y streaming
`node-web` → `node-repository`	HTTP :8084	Carga y compartición de archivos
`node-web` → `node-monitoreo`	HTTP :3000	Tableros de monitoreo
`node-sync` → `node-repository`	NFSv4 :2049	Lectura y escritura
`node-media` → `node-repository`	NFSv4 :2049	Solo lectura
`node-repository` → `node-soap`	LDAPS :636	Traducción de uid/gid para NFSv4
`node-rmi` → `hpc-master`	RMI/JRMP :2099, objetos en :2100	Envío del trabajo y callback de estado
`hpc-master` → `hpc-worker`	SSH :22 y MPI sobre TCP	Ejecución paralela
`hpc-master` → `node-repository`	NFSv4 :2049	Código, datos y resultados
Clúster → `node-monitoreo`	HTTP :9100	Métricas

> **Plan de puertos.** El rango `80xx` es propio del proyecto para los servicios de aplicación. Los puertos `636` (LDAPS), `2049` (NFSv4), `9100` (exporter) y `3000` (Grafana) son estándares de cada producto. El registro RMI se trasladó del `1099` por defecto al `2099`, y los objetos remotos se fijan en el `2100`.

5. Decisiones de arquitectura

Cada decisión se documenta junto con la alternativa que se descartó y el motivo.

D1 — node-web como única puerta de los clientes web

La interfaz no habla con SOAP, RMI ni el repositorio. *Alternativa descartada:* que el navegador llame a cada servicio directamente; se descartó porque expone la superficie completa del sistema y obliga a resolver CORS y validación de sesión en cinco lugares distintos.

D2 — SOAP y LDAP en la misma máquina

La consulta al directorio viaja por loopback. *Alternativa descartada:* directorio en máquina propia; la integración ahorra un servidor y elimina un salto de red en la operación más frecuente del sistema.

D3 — Almacenamiento con doble vía de acceso

NFS para los servicios de backend, HTTP para el navegador. *Alternativa descartada:* montar NFS también en `node-web`; se descartó porque situaría el camino de escritura de datos de usuario sobre un sistema de archivos de red gestionado desde JavaScript, y porque la lógica de compartición no se puede expresar únicamente con permisos de archivo.

D4 — Medios separados del repositorio

Aísla el consumo de CPU de FFmpeg. *Alternativa descartada:* alojar todo en `node-repository`; se descartó por contención de recursos sobre el nodo más crítico de la arquitectura.

D5 — El clúster tiene un único punto de entrada

Solo `node-rmi` se comunica con `hpc-master`. Es lo que refleja la Figura 1, donde el clúster mantiene una sola línea y ésta llega a los servidores de aplicación.

D6 — Respaldo con llave pública y privada custodiada fuera

Satisface el requisito de cifrado asimétrico y protege contra el compromiso del propio servidor de datos.

D7 — La malla Grid queda fuera del alcance

El enunciado plantea integrar equipos externos y dispositivos IoT a la malla. `gestor-cluster.jar` se diseña para admitir nodos externos, pero en este período únicamente se integran las máquinas del laboratorio. Queda registrado como limitación explícita del alcance, no como omisión.

6. Lo que el diagrama asume y no dibuja

Esta sección recoge los supuestos que más se malinterpretan al leer el diagrama por primera vez.

Los tres clientes ofrecen los mismos servicios

El diagrama no muestra pantallas porque las pantallas no son arquitectura. Que el móvil sincronice por gRPC y consulte el portal por HTTPS ya implica que puede subir y consultar todo el contenido. **La transparencia no proviene de que los clientes sean idénticos, sino de que escriben sobre el mismo Home.** Si el móvil sube una fotografía, `syncd` la escribe en `/srv/home/<usuario>` y `album-svc` la lee de allí cuando el usuario abre el álbum desde el navegador o desde otro equipo. Es el mismo modelo de un servicio de almacenamiento en nube comercial: tres interfaces distintas sobre un único estado compartido.

Todas las máquinas exponen exporter en el puerto 9100

Se dibuja una sola ruta hacia `node-monitoreo` para no saturar el diagrama, pero el monitoreo cubre toda la infraestructura, no únicamente el clúster.

Todos los nodos del clúster montan /srv/home

Aunque la flecha del diagrama salga del nodo maestro, el montaje aplica a la totalidad del clúster por exigencia de MPI.

El análisis de vulnerabilidades no es un componente desplegado

El enunciado lo exige sobre la aplicación web en ambiente de producción, con los niveles de riesgo asumidos documentados. Se registra como evidencia aparte, no como un nodo del diagrama.

7. Puntos abiertos

Decisiones pendientes que deben cerrarse antes de avanzar en el código.

1. ¿Cómo valida syncd la credencial que recibe?

Actualmente el agente se autentica contra SOAP y luego utiliza gRPC, pero `node-sync` no consulta a nadie. Existen dos caminos:

- **Token autocontenido:** `syncd` verifica la firma con una llave pública, sin preguntar. Es rápido, pero **no permite revocar** la sesión antes de que expire.
- **Token opaco con introspección:** `syncd` pregunta a SOAP si la sesión sigue vigente. Permite revocación inmediata a cambio de una llamada adicional por petición.

Mientras no se decida, el artefacto del cliente debe llamarse `credencial.local` y no `token.jwt`: el nombre de un archivo no puede adelantar una decisión que aún no se ha tomado.

2. Conexión del móvil con el portal

Falta la ruta `Android Mobile → node-web : HTTPS :443`. Se acordó que los tres clientes ofrecen los mismos servicios; sin esa línea el diagrama afirma que el móvil únicamente sincroniza.

3. Control de versiones y conflictos en node-sync

El requisito establece que la sincronización mantiene una versión del recurso en el servidor y que cualquier cambio en una copia se refleja en todas. Hace falta un artefacto donde residan las versiones y una política explícita de resolución de conflictos.

4. ¿Los nodos del clúster son clientes LDAP?

Si montan NFS, necesitan resolver uid y gid igual que el repositorio. Con LDAP, los trabajos se ejecutan con la identidad real del usuario y los resultados quedan en su Home con los permisos correctos. Con una cuenta de servicio única no hay configuración adicional, pero los resultados quedan con propietario equivocado.

5. Sincronización de reloj

Instalar `chrony` en las siete máquinas virtuales y en los nodos del clúster, apuntando al mismo servidor NTP. Con tokens que expiran y registros correlacionados entre nueve máquinas, un desfase de reloj produce fallos de autenticación intermitentes muy difíciles de diagnosticar.

8. Frentes de trabajo

Seis frentes, un responsable por frente. La interfaz entre módulos se acuerda **antes** de empezar; dentro de su frente, cada integrante desarrolla con libertad.

#	Frente	Alcance
1	Infraestructura y repositorio	Máquinas del CCA, red, NFSv4, cuotas y respaldo GPG
2	Identidad	OpenLDAP, SOAP en PHP, segundo factor y análisis de vulnerabilidades
3	Portal web	Node.js, interfaz gráfica e integración con Shared File
4	Clúster	RMI, `job-runner`, `hostfile` y MPI en el laboratorio
5	Sincronización	`syncd` en Go y agente de escritorio
6	Móvil y medios	Aplicación Android, álbum, streaming y monitoreo

Los frentes 5 y 6 son los más exigentes; el frente 1 se libera pronto y puede reforzarlos. **Aunque cada integrante trabaje por separado, todos deben poder explicar el sistema completo**, no únicamente su módulo. Este documento es el mínimo común que debe manejar el equipo.

9. Riesgos conocidos

Riesgo	Impacto	Mitigación
El CCA entrega menos máquinas de las previstas	Alto	Fusionar en este orden: `node-media` dentro de `node-repository`, y luego `node-monitoreo` dentro de `node-web`
Puerto aleatorio de objetos RMI bloqueado por el firewall	Alto	Fijar el puerto `2100` desde el primer commit
Traducción de identidades sin LDAP: permisos como `nobody:nogroup`	Alto	Configurar los nodos como clientes del directorio antes de montar NFS
`node-repository` es punto único de fallo	Medio	Asumido y documentado; la replicación queda fuera del alcance del período
Transcodificado saturando el nodo de medios	Bajo	Procesar en diferido y con prioridad reducida

---

Este documento acompaña al diagrama de despliegue y al diagrama de componentes del proyecto. Debe actualizarse cada vez que se cierre uno de los puntos abiertos de la sección 7.